Hexagons are the bestagons: A Novel Neural Network Topology

Daniel Lopez, Dr. Cezareo Rodriguez

May 2026

Abstract

We propose and establish baselines for a novel, albeit impractical neural network topology that departs from traditional Cartesian grid layouts of neurons and connections and employs a hexagonal lattice enabling neighboring networks with overlapping weights, Hexagonal Neural Networks. Inspired by extreme behavior compartmentalization behavior disorders observed in the general populace, this network structure focuses on the influence of directional training on overlapping weights. Early baselines reveals new learning behaviors through loss and accuracy iterations and consistent improvement from single direction propagation which has widespread usage in modern day commercial deep learning structures.

1 Introduction

1.1 Gap in the Literature

Modern day neural networks, such as the classic multilayer perceptron (MLP), are traditionally structured as a unidirectional chain of connected layers, where each layer only connects to the next in sequence. This imposes a simple ordered sequence of transformations on the input data, where each layer defines a projection from one dimensional space to another, along with some activation function. In the context of the latent space, the transformations represent a hierarchical structure of feature abstractions that depend on the ordered sequence of layers.

Through these variations, many inductive biases have been successfully encoded such as convolutional locality, invariance to translation, rotation, scaling, and other geometric transformations. While many variations of this base model have been iterated on with respect to shared weights, activation functions, and other hyperparameters, the core architectural ordering of the network remains the same and only a few deviations encode geometric transformations of the data, such as 2D or 3D convolution layers popularized in computer vision tasks. Convolutional Neural Networks (CNNs) are a prime example of this, where the convolutional filters are rotated for angular invariance of features in the context of image recognition using parameter sharing in a abstraction we commonly refer to as filters or kernels.

1.2 Hexagonal Neural Networks

Let's imagine an architecture that uses a 2D topological arrangement of layers, but instead of having a single direction of propagation, it has multiple directions of propagation. They would also use parameter sharing, but in a more flexible way; they would be using the rotation of the entire network itself to ask if directionality and rotational interpretation can be used as an architectural constraint to encode more complex inductive biases through disruptive cross training methods and techniques, if any exist.

Moreover, human cognition and artificial intelligence are known to face common challenges, such as managing knowledge across diverse tasks without losing prior knowledge gain. This is known as catastrophic forgetting, where a network trained on one task is unable to perform well on a different task. This is a known challenge in artificial intelligence, and it is a known challenge in human cognition.

Here, we introduce Hexagonal Neural Networks (HexNNs) that may generalize MLP into a rotation-

ally symmetric, multi-directional structure.

HexNNs replace the standard 1D layer stack with an initial 2D topological arrangement made to resemble regular hexagons. In any one direction, they may still be intepreted as a unidirectional chain of connected layers. This allows for the same node set may be traversed in a different direction, and the network may be interpreted as a different network in that direction with some overlap of weights between the two networks.

However, this adds a constraint to the data that may be interpreted as the input and output spaces being the same. This would not necessarily be well suited for machine learning tasks such as classification that require a fixed input and output space, although future work may explore this further.

Given a size parameter, n, the network has equal n length input and output layers, with hidden layers symmetrically increasing and decreasing to $2n - 1$. Crucially, each node in layer i is connected to all nodes in layer $i + 1$, but unlike MLPs, this bipartite adjacency is true for all three cardinal directions of a hexagon. This allows for rotation-based reinterpretation of the same input data.

This design enables six distinct orientations of the same network, each corresponding to a 60° rotation of the hexagonal structure. We define each orientation as a subset of the network, H_i, and note that forward propagation of any H_i is equivalent to backwards propagation of its corresponding opposite network, $H_{(i+3) \bmod 6}$.

1.3 Open Questions

As we can see, this new change in architecture may have some interesting implications for the training of the network. What are the implications of training the network in one direction, and how does that impact the training of the network in other directions? What effects will this have on the performance of the network in other directions? Is there an optimal sequencing of training with intermitten rotations that can yield lower overall loss in any one network or an mixture of experts (MoE) approach that can yield lower overall loss than the MLP equivalent?

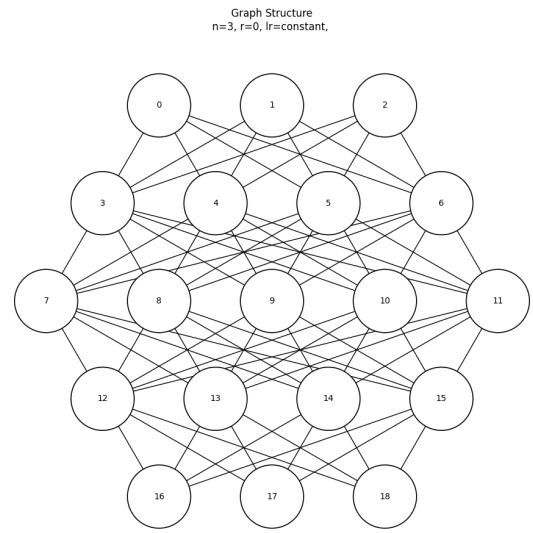

Figure 1: A Hexnet with N=3 and default rotation H_0

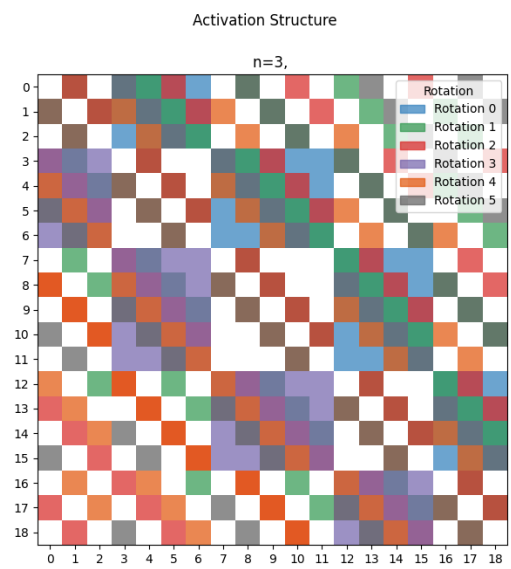

Figure 2: The Multi Activation Structure showing an all to all adjacency matrix with each distinct direction in a different color

1.4 Contributions

In this paper, we present a memory efficient implementation using over padded state vectors and orientation specific adjacency matrices. We also establish baselines for HexNNs and their equivalent MLPs and compare the performance of the two architectures across various datasets and hyper parameter configurations.

We also present the inductive bias: [blank] Unlike MLPs, HexNNs are able to present [blank] which can tell us if the data is [blank]. This could be useful for [blank] tasks where modern day neural networks are not able to perform well.

2 Background

2.1 MLPs and Layered Function Approximation

2.2 Symmetry and Equivariance in Neural Networks

2.3 Graph Neural Networks

2.4 Parameter Sharing, Ensembles, and Multi-Head Models

2.5 Continual Learning and Interference

2.6 Cognitive Motivation

Ground breaking architectural models that have been proposed are able to uncover different inductive biases that are not possible with the traditional MLP architecture using natural design as motivation. CNNs, for example, discovered feature invariance after modeling the convolutional kernals after the visual cortex and its neurophysiological design. RNNs and LSTMs resemble the neurophysiological self looping design of the hippocampus and its role in memory and spatial navigation and this was able to introduce the concept of recurrent connections and the idea of memory cells in artificial intelligence. So consider the phenomenon found in people who have a disassociate identity disorder (DID), where memories and personalities are split between identities despite having the same brain. To borrow speculative language from Douglas Hofstadter, we can imagine a network toplogy that is able to hold different representations of the same data in the same hardware and each "strange loop" working as its own coherent identity but potentially affecting the learned experiences with other identities.

To be clear, this paper does not model DID or aim to benchmark DID like symptoms in a clear method like the research proposed by [blank]. DID was the human based inspiration given its similar characteristics to extremely compartmentalized behavior, something not usually found in highly emergent cognitive systems, such as humans. This inspiration also comes from the primary author's own trauma with a mother with split identities, and a father with hemiplegia after a hemoraggic stroke impaired his ability to move his left side. The cognitive analogy is strictly heuristic, not empirical.

3 Implementation

3.1 Topological Definition

A one directional view hexagonal neural network is a multilayer perceptron network whose layer sizes increase and then decrease symmetrically, forming a hexagon-like outline if it were to be drawn in 2D. As expected, all neurons in any given layer are fully connected to all neurons in the subsequent layer. However, due to the hexagonal nature of the framework, there are three distinct directions in which one could perform traditional forward and backward neural network propagation. This renders the structure to contain a unique corresponding arrangement of connections and shared weights given the direction.

These shared weights may influence the internal network's representation of data in their high-dimensional embedding from one propagation direction to another in strange ways. The shared weights of, say, the first semantic layer of H_1 will always have at least one shared weight with H_2 (the relative left-most neuron from H_2's point of view) up until the halfway point of the network. This overlap is small

but non-neglible as these weights are now subject to the gradient descent paths of two distinct target local optimas. Experimentally, we actually see equivalent or better another local optimum, outperforming the two original networks had they been started with the same random seed patterns for their respective weights across the benchmark set (detailed below). Since each layer represents a layer of abstraction away from the input reality towards the target reality, it could be speculated that any one layer's abstraction has a cross abstractive coherence bias on another's direction and thus representation.

3.2 Mathematical Definitions

The layer sizes of a HexNet of a parameter size n follow an explicit pattern:

$$L = [n, n+1, ...2n-1, ..., n+1, n]$$

with a $2n-1$ total layers as well. The total number of nodes in a network, however, is:

$$T = \sum_{k=0}^{2n-2} (n + \min(k, 2n-2-k))$$

3.2.1 Weight Indexing and Layer Sets

To be explicit, if we visualize the network with the layers centered and stacked vertically (i.e. with the smallest layers at the top and bottom, widest in the middle), then we have the following directions:

- H_0**:** Top-to-bottom layers (default hexagonal structure)
- H_1**:** Top-right to bottom-left diagonal
- H_2**:** Bottom-right to top-left diagonal
- H_3**:** Bottom-to-top layers (reverse of H_0)
- H_4**:** Bottom left to top-right diagonal
- H_5**:** Top left to bottom-right diagonal

3.2.2 Global W

For mathematical simplicity, we employ a global weight matrix W, a adjacency matrix of size TxT that can represent all possible connections regardless of rotation. Given that H_0 is directionally opposite H_4, they would share exactly the same weights. While empirical trials on MLPs show that bidirectional training on the same weights show catastrophic loss, (find reference), we could theoretically use different cells to represent that as a separate network and still maintain directional-abstraction bleeding between immediately neighboring dimensions. If this were the case, then W would not be symmetric.

3.2.3 Directional W

We can represent the static MLP view of any arbitrary rotation, $r \in 0, 1, 2, 3, 4, 5$ as W_i with a mask over W enforcing a layered feedforward structure.

For the next few mathematical definitions, let the **Vertical Direction** orientation of the network, H_1, be the assumed direction. Furthermore, let's assume the numbering of the network to start from the top left index, traverse left to right, and then top to bottom. Let:

- $A^i \in \{0,1\}^{T \times T}$ be the binary adjacency matrix for arbitrary direction H_i
- For node index $i \in [0, T-1]$, let layer(i) map it to its layer number $l \in [0, N_L - 1]$
- S_l be the set of indices in layer l

Then, the H_1, the only non-zero entries in A are defined as:

$$A_{ij} = \begin{cases} 1 & i \in S_l, j \in S_{l+1}, \text{ for some } l \in [0, N_L - 2] \\ 0 & \text{otherwise} \end{cases}$$

A symptom of this definition for H_1 is a block structure that gets created inside A^1. Each block $(l, l+1)$ is a dense matrix of ones with size $|S_l| \times |S_{l+1}|$ Thus, A^1 can be visualized as:

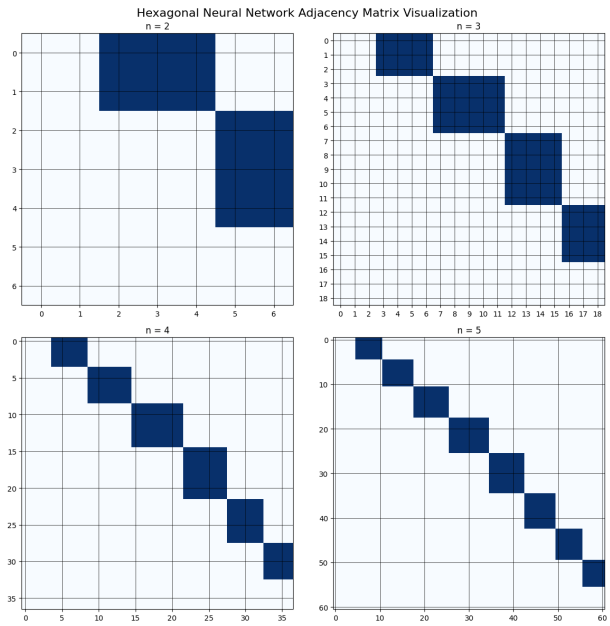

Figure 3: Adjacency Matrix showcasing the Block structure for H_1

$$A^1 = \begin{bmatrix} 0 & B_0 & 0 & \dots & 0 \\ 0 & 0 & B_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & B_{N_L-2} \\ 0 & 0 & \dots & 0 & 0 \end{bmatrix}$$

where every $B_k \in \{0,1\}^{|S_k| \times |S_{k+1}|}$ is a matrix of all 1s. We can see this pattern hold as n grows for H_1 in 3

3.2.4 Rotations

As previously noted, properties such as layer sizes and layer weights for H_i will be the reverse for the geometric opposite network, $H_{(i+3) \bmod 6}$, and therefore, adjacency matrices, weight matrices, and other relevant matrices can be reused.

3.3 Traditional Neural Network Operations

The connections between nodes in any neural network serve as weights, and thus the adjacency matrix A^i can be $W^i \in \mathbb{R}^{T \times T}$ where all values that are not zero as previously defined are now a real number.

Thus, for forward propagation, you can just iteratively compute:

$$x^{t+1} = f(W^T \times x^t)$$

where f is a typical activation function for neural networks, and x is the padded form of the input vector. This padding brings x from n dimensions to T dimensions for ease of multiplication with W.

3.3.1 Example Flow with $n = 2$

In a network with $n = 2$,

- The layer sizes are $[2, 3, 2]$
- The total number of nodes is $T = 7$
- The Weight and Adjacency matrices $A, W \in \mathbb{R}^{7 \times 7}$
- The padded input vector x is

$$x = \begin{bmatrix} x_0 \\ x_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- Computation is simply:

3.4 Network Variance

If we interpret six co-dependent multi layer perceptrons as an ensemble with parameter tying, here are a few variants that we could try finding baselines within the guidance of ensemble based learning.

We will establish some baseline models, and their expected outputs. Then we will try five variants of

these baselines that are based on exploits and manipulations that we can do exclusively because of hexagonal nature of the graph. To these five, we will then be testing some orthogonal knobs. We will establish what are the independent and dependent and testing variables that we will be fine tuning and sweeping through to test.

First, let's establish some baselines.

3.4.1 Baselines

A single model, without sharing, standard training Deep ensemble, six separate networks, no sharing, independent seeds. Snapshot ensemble / SWA: 1 network, multi snapshot averaging MC-Dropout: 1 net, inference-time dropout

3.4.2 Variants 1 - 5: Hex Ensemble Variants

Variant 1: Hard Rotation Sharing: One backbone tensor bank, each member 'applies a hex rotation' to filters adjacency before its forward. Only small member-specific heads (and optional tiny offsets) are unique. Combines group equivariance with ensembling

Variant 2: Soft sharing via low-rank delta (LoRA-style) Shared backbone θ; each member has $\Delta_i = A_i B_i$ (rank-r) on selected layers. Control r to trade diversity vs. parameters Optimal: permember BN/Layer Norm

Variant 3: Masked weight reuse Shared tensor bank W; each member uses a binary/ternary mask M_i (learned or fixed) such that $W \circ M_i$ Encourages orthogonal subspaces with negligble memory

Variant 4: Shared backbone, member heads only Classic multi head: 100 percent sahred feature extractor; six heads (Acts as lower bound on diversity)

Variant 5: Optimizer state-sharing vs isolatioin Same weights but either (i) one optimizer for all members (cotraining) or (ii) per member optimizers on their delta/heads only.

4 Result

4.1 Base Case

Baselines for H_0 have been created where loss function, activation functions, and n are variable, and we can measure their performance with regression metrics such as scalar loss (according to the respective loss function), accuracy (root mean square deviation, as is standard for regression based data sets) and R^2 (the coefficient of determination).

4.2 Simulated Data Setup

I did things here (identity dataset, linear dataset, chat gpt outputted)

4.3 Real Data Sets

I did things with real data here